DataMate: tài liệu học tập

Từ câu hỏi tiếng Việt đến ngữ cảnh cho mô hình: những gì đã được xây dựng, vì sao, và cách tự kiểm chứng

- Dự án: **BANKAI-DATA** (Zalo AI Challenge 2026 - DataMate), hỏi đáp tiếng Việt trên dữ liệu bảng của các cửa hàng.
- Phạm vi tài liệu: hai đợt công việc trong phiên làm việc này (đợt 1: giao diện chung, chuẩn hóa có vị trí, viết tắt; đợt 2: thời gian và đối tượng).
- Cuối tài liệu có bảng tra file, các lỗi gặp phải cùng bài học, và bài tập tự làm.

1. Bức tranh tổng thể

Hệ thống nhận một câu hỏi tự nhiên (ví dụ: *"hnay dthu cua sp bán dc từ 5k trở lên tuần trc là bnhiu"*) cùng dữ liệu của một cửa hàng, rồi nhờ mô hình ngôn ngữ viết SQL để trả lời. Mô hình hay sai ở những việc cần tính toán chính xác: ngày tháng, viết tắt, tên sản phẩm gõ sai. Vì vậy phần **hiểu câu hỏi** được làm bằng code thường (xác định, kiểm thử được), rồi đưa kết quả vào prompt dưới dạng các *mục gợi ý*.

```
câu hỏi
   |
   v
Query (văn bản NFC + bản chuẩn hóa có vị trí + ngày as_of)
   |
   v   các Resolver chạy lần lượt theo thứ tự ưu tiên
money -> lunar -> time -> shift -> basis -> coverage -> slang -> entity -> ambiguity
   |       mỗi resolver trả về các Finding (có vị trí ký tự)
   v
understand(): bỏ Finding nào đè lên ký tự đã bị resolver mạnh hơn chiếm
   |
   v
build_context(): gom Finding thành các mục của prompt
   (RESOLVED TIME RANGES, AMOUNTS, SHIFTS, TIME BASIS, DATA COVERAGE,
    ENTITY HINTS, AMBIGUOUS ENTITIES, ABBREVIATION HINTS)
   |
   v
mô hình viết SQL  ->  chạy SQL  ->  đáp án
```

> **Nguyên tắc xuyên suốt:** mọi thứ có thể tính bằng code thì tính bằng code, mô hình chỉ nhận kết quả. Mỗi bước đều có test để biết khi nào bị hỏng.

2. Đợt 1: nền móng

2.1 Giao diện chung: Finding và Resolver (understand/base.py)

Trước đó ba phần (thời gian, viết tắt, thực thể) là ba hàm rời rạc, mỗi hàm trả về một kiểu dữ liệu khác nhau. Việc đầu tiên là đưa chúng về **một giao diện** để có thể ghép, đổi thứ tự, và so vị trí với nhau.

```
@dataclass(frozen=True)
class Finding:
    resolver: str      # "time", "lunar", "money", "slang", "entity"...
    kind: str          # loại con: "month", "abbr", "amount"...
    start: int         # đoạn ký tự trong câu hỏi, end không tính
    end: int
    text: str          # đúng đoạn Query.text[start:end]
    line: str          # một dòng để đưa vào prompt
    value: Any = None  # kết quả có cấu trúc (Span, số tiền, Hint...)

class Resolver(ABC):
    name: str
    section: str          # tên mục trong prompt
    claims: bool = True   # True: chiếm các ký tự mà nó tìm thấy
    def resolve(self, q: Query) -> list[Finding]: ...
```

Hàm **understand(q, resolvers)** chạy các resolver theo thứ tự ưu tiên (đầu danh sách mạnh nhất). Finding nào chồng lên ký tự mà resolver mạnh hơn đã chiếm sẽ bị bỏ. Resolver có `claims=False` (như thực thể) chỉ để gợi ý, không chiếm và không bị chặn.

```
def understand(q, resolvers):
    kept, claimed = [], []
    for r in resolvers:
        q.found = kept          # resolver sau đọc được kết quả của resolver trước
        mine = [f for f in r.resolve(q)
                if not (r.claims and any(f.overlaps(c) for c in claimed))]
        kept += mine
        if r.claims:
            claimed += mine
    return sorted(kept, key=lambda f: (f.start, f.end))
```

Điều cần học: đặt các thành phần sau một interface nhỏ giúp thêm tính năng mà không sửa nơi gọi. Mình giữ logic cũ của `timeparse.py` và `lunar.py` nguyên vẹn, chỉ bọc chúng bằng `TimeResolver` và `LunarResolver` (adapter). Nhờ vậy 107 test cũ vẫn là bằng chứng rằng hành vi không đổi.

2.2 Chuẩn hóa có vị trí (normalize.py: lớp Normalized)

Hàm `norm()` cũ bỏ dấu, hạ chữ thường, gộp khoảng trắng. Sau khi chuẩn hóa, ta mất thông tin "ký tự này nằm ở đâu trong câu gốc". Muốn biết hai phát hiện có đè lên nhau không thì phải có vị trí. **Normalized** lưu một mảng `origin`: ký tự thứ i của văn bản đã chuẩn hóa đến từ ký tự nào của câu gốc.

```
nz = Normalized("  Doanh   thu  THÁNG   này  ")
i = nz.text.index("thang nay")
s, e = nz.to_raw(i, i + len("thang nay"))
nz.raw[s:e]            # -> "THÁNG   này"  (đúng chữ người dùng viết)
nz.words()             # -> [Word("doanh",...), Word("thu",...), ...] kèm vị trí
```

- Thuộc tính quan trọng: `Normalized(q).text == norm(q)`, đã kiểm trên cả 3.000 câu train.
- Câu hỏi được đưa về dạng NFC trước (chữ có dấu thành một ký tự), nên mọi vị trí trong hệ thống đều trỏ vào cùng một chuỗi.
- Ví dụ "không giẫm lên nhau": *"5k"* là số tiền 5.000, không phải viết tắt "k"; *"năm 2025"* nằm trong *"Tết năm 2025"* thì để resolver Tết lấy.

2.3 Viết tắt và slang.json

Khai thác dữ liệu thay vì đoán. Script `scripts/mine_slang.py` liệt kê các từ ngắn, hiếm, chỉ có chữ ASCII và *không* phải dạng bỏ dấu của một từ có dấu nào trong corpus. Ví dụ "thang" bị loại vì corpus có "tháng"; "dthu" và "bnhiu" được giữ lại. Sau đó **duyệt tay** và ghi vào `understand/slang.json` (không viết cứng trong code).

Từ	Nghĩa	Bằng chứng trong train
dthu	doanh thu	482 lần
bn, bnhiu	bao nhiêu	211 và 184 lần
dc / đc	được	229 lần
vs, z, j	so với, vậy, gì	65, 54, 6 lần
hnay, hqua, trc, th	hôm nay, hôm qua, trước, tháng	60, 48, 60, 37 lần
k	**không** (không phải nghìn)	5 lần, cả 5 đều là "không"
hd	**không đưa vào**	4 lần, đều là tên sản phẩm (Telfast HD)

Luật theo ngữ cảnh. "k" chỉ là nghìn khi đứng ngay sau một con số. Phần này nằm ở mục `after_number` của slang.json và do `MoneyResolver` xử lý:

```
"after_number": {
  "k":  {"expand": "nghìn", "mult": 1000},
  "tr": {"expand": "triệu", "mult": 1000000}
}
# "5k" -> 5000    "2,5tr" -> 2500000    "1.500k" -> 1500000
# "tháng 5 k kê đơn", "7/2025 k" -> KHÔNG phải tiền, "k" là "không"
```

Money chạy **trước** slang, nên chữ "k" của "5k" đã bị chiếm và slang không đọc lại lần hai. Một chữ "k" đứng riêng sẽ được hiểu là "không".

3. Đợt 2: thời gian

Rulebook §2 có nhiều quy tắc thời gian mà code cũ chưa phân biệt. Ba bổ sung, đều là resolver mới:

3.1 "Phát sinh" hay "tình trạng đến hết ngày" (basis.py)

Hai loại câu hỏi tính khác nhau hoàn toàn:

Loại	Ví dụ	Cách tính
FLOW (phát sinh)	doanh thu hôm qua; số đơn tháng 3/2025	chỉ các bản ghi **nằm trong** khoảng ngày
STOCK (tình trạng)	tính đến hết ngày 15/02, bao nhiêu khách còn nợ; hết hôm nay còn mấy lon bia	cộng **mọi** bản ghi từ ngày đầu của dữ liệu đến hết ngày đó

```
_STOCK = re.compile(r"\bcon (no|thieu|ton|hang|lai|bao nhieu|may)\b|\bton kho\b|...")
_FLOW  = re.compile(r"\bghi no them\b|\btra no\b|\bthu hoi\b|...")   # luôn thắng

def stock_cue(text):
    if _FLOW.search(text): return None
    return _STOCK.search(text)
```

Bài học thiết kế: cụm "đến hết hôm nay" xuất hiện cả trong câu phát sinh ("từ đầu tháng đến hết hôm nay, doanh thu..."), nên không dùng nó làm tín hiệu STOCK. Tín hiệu đáng tin là *từ chỉ số còn lại* (còn nợ, còn tồn, tồn kho...). Đây là heuristic dựa trên quy tắc; train không có nhãn FLOW/STOCK nên mình chưa đo được độ chính xác.

3.2 Ca sáng, ca chiều (shift.py)

"Ca sáng" thành 06:00 đến 13:59, "ca chiều" thành 14:00 đến 21:59, theo giờ tạo đơn. Resolver còn liệt kê các cột giờ trong dữ liệu (ingest tách mỗi timestamp thành `cột` và `cột_time`). Khi so ca chiều với ca sáng, mốc so sánh là ca sáng.

3.3 So khoảng ngày với phạm vi dữ liệu (coverage.py)

Rulebook §7: nếu kỳ được hỏi nằm **hoàn toàn ngoài** khoảng ngày có dữ liệu, đáp án là *unanswerable*, không phải 0. Nếu nằm một phần thì tính trên phần có dữ liệu. Muốn làm vậy phải biết shop có dữ liệu từ ngày nào đến ngày nào: hàm `event_date_range` trong ingest.py lấy min và max của các cột ngày **giao dịch**, bỏ ngày sinh, hạn dùng, ngày áp dụng giá, ngày tạo danh mục.

Chỉ số (3.000 câu train, 70 câu unanswerable)	Kết quả
Cảnh báo "ngoài phạm vi" khớp đáp án unanswerable	14 / 14 (độ chính xác 100%, không báo nhầm)
Số câu unanswerable được bắt	14 / 70 (phần còn lại là shop không ghi loại thông tin)

> **Cách đo (đáng học):** lấy đáp án có sẵn làm "đáp án vàng", chạy resolver trên mọi câu hỏi, rồi đếm đúng, báo nhầm, và bỏ sót. Với tính năng "cảnh báo", báo nhầm tệ hơn bỏ sót, nên ưu tiên độ chính xác.

4. Đợt 2: đối tượng (ground.py)

4.1 Phân loại theo loại thực thể

Mỗi cột chữ được gán một loại: **product, customer, staff, category, payment** hoặc other. Gán theo tên cột, tên bảng (cột "ten" trong bảng nhan_vien là nhân viên) và với payment còn dựa vào giá trị (Tiền mặt, Chuyển khoản, Momo...). Các luật được thử theo thứ tự, luật đầu khớp thì thắng.

```
classify("sp", "ten_hang")              -> "product"
classify("don", "ten_khach_hang")       -> "customer"
classify("sp", "loai_san_pham")         -> "category"
classify("don", "so_luong_san_pham")    -> "other"     # đây là số lượng, không phải sản phẩm
classify("nhan_vien", "ten")            -> "staff"
```

4.2 Mơ hồ thì liệt kê lựa chọn

Nếu một cụm khớp nhiều giá trị gần nhau trong cùng một cột (chênh nhau chưa tới 0,1 điểm) và không có giá trị nào khớp hoàn toàn, resolver ghi mục **AMBIGUOUS ENTITIES** và liệt kê các lựa chọn để mô hình chọn theo phần còn lại của câu hỏi:

```
[product] "bia tiger" fits 3 values of sp.ten_hang:
  'Bia Tiger lon' | 'Bia Tiger chai' | 'Bia Tiger Bạc lon'
  - the question does not say which; use the rest of the question to choose,
    otherwise cover all of them
```

Cùng một cụm khớp hai loại khác nhau (ví dụ "cà phê" vừa là sản phẩm vừa là nhóm hàng) cũng được báo, nhưng chỉ khi hai đoạn khớp trùng hẳn nhau. Nếu sản phẩm cụ thể hơn ("Bia Tiger lon" so với nhóm "bia") thì không coi là mơ hồ.

4.3 Chỉ mục n-gram thay cho quét difflib

Khi một từ trong câu hỏi gõ sai ("thep" thay vì "thép"), code cũ gọi `difflib.get_close_matches` trên **toàn bộ từ vựng** (hàng chục nghìn từ) cho mỗi từ lạ. Chỉ mục mới lập bảng "cặp ký tự liền nhau (bigram) → các từ chứa nó":

```
def closest(self, word):
    # 1. lấy các từ chia sẻ bigram với word (tra bảng, không quét)
    # 2. lọc theo độ dài (tỉ số khớp tối đa = 2*min/(la+lb) phải >= 0.84)
    # 3. chỉ chấm điểm SequenceMatcher cho 40 ứng viên đầu
    # 4. trả về từ có tỉ số cao nhất >= 0.84 (hòa thì lấy chuỗi lớn hơn, y như difflib)
```

Shop (từ vựng)	difflib	n-gram	Kết quả khác nhau
biz_103 (18 nghìn từ)	0,10 s	0,005 s	0
biz_130 (46 nghìn từ)	0,29 s	0,003 s	0
biz_122 (60 nghìn từ)	0,38 s	0,009 s	0

Điểm cốt lõi: **giữ nguyên hợp đồng** (cùng đầu vào thì cùng đầu ra) và kiểm bằng cách so với bản cũ trên dữ liệu thật, rồi mới tin vào tốc độ.

4.4 Hai sửa đổi rút ra từ dữ liệu thật

- **Che phần đã bị chiếm.** Hàng "Tổng tháng 2/2025" trong dữ liệu khớp với cụm "tháng 2/2025" của câu hỏi, sinh cảnh báo mơ hồ giả. Giờ các phần của câu đã bị resolver thời gian/tiền/ca chiếm được thay bằng khoảng trắng trước khi tìm thực thể.
- **Gộp bảng theo tháng.** Shop tách dữ liệu thành nhiều bảng theo tháng (donhang_chitiet_04_2025, _05_2025...). Cùng một cột và giá trị chỉ hiện một dòng kèm "(+11 same-named tables)"; trước đó 10 dòng gợi ý toàn là "lon" và "bao" che mất sản phẩm đúng.

5. Lỗi gặp phải và bài học

Chuyện gì xảy ra	Nguyên nhân	Bài học
"tháng 7/2025" bị hiểu thành tháng 7/2026	pattern cũ chỉ nhận "tháng 7 năm 2025", dạng có dấu / bị bỏ qua rồi tự suy ra năm. 1.256/3.000 câu train dùng dạng này	Test cũ né lỗi bằng .replace(...). Khi một tính năng mới phụ thuộc vào một thành phần cũ, kiểm tra thành phần cũ trước. Đã sửa pattern tháng và quý.
Tính năng mới chỉ bắt 3/70 ca	hậu quả của lỗi trên	Đo trước khi tin. Con số thấp là tín hiệu có lỗi ở nơi khác.
Gợi ý "mơ hồ" nhiễu ('1 lon' | '2 lon'...)	cột số lượng bị coi là sản phẩm	Chạy thử trên dữ liệu thật, đừng chỉ dựa vào test tổng hợp.
Test CSV một cột bị đọc sai (cột thành "ha")	bộ đoán dấu phân cách của _read_csv	Test dùng CSV hai cột. Đây là điểm yếu của ingest, nên xem lại sau.
biz_101 và biz_107 không có phạm vi dữ liệu	ngày ghi "1 thg 6 2025" (biz_101) và số serial Excel 45717 (biz_107) không được nhận là ngày	Lỗi ingest có sẵn. Cảnh báo im lặng thay vì báo sai.
git stash không hoàn tác được code	thư mục src/ chưa được git theo dõi, stash bỏ qua file untracked	Muốn so với bản cũ phải sao lưu thủ công hoặc commit trước.

6. Cách tự chạy và kiểm tra

```
cd /Users/sunix/Documents/BANKAI-DATA
python -m pytest -q                          # 133 test
python -m pytest tests/test_understand.py -v
python -m pytest tests/test_basis_coverage_entity.py -v

python scripts/mine_slang.py train/questions.jsonl | head -60   # tìm viết tắt mới
```

Thử tay một câu:

```
python - <<'EOF'
import sys; sys.path.insert(0, "src")
from datetime import date
from datamate.understand import *

q = "hnay dthu cua sp bán dc từ 5k trở lên, hàng k kê đơn tuần trc là bnhiu"
for f in understand(Query(q, date(2026, 3, 4)),
                    [MoneyResolver(), TimeResolver(), SlangResolver()]):
    print(f.resolver, f.start, f.end, repr(f.text), "->", f.line)
EOF
```

Muốn thêm một từ viết tắt: sửa `src/datamate/understand/slang.json` (khóa viết thường, không dấu), rồi chạy pytest. Test *keys are accent-free lowercase* sẽ báo nếu bạn gõ sai định dạng.

7. Bảng tra file

File	Vai trò
understand/base.py	Finding, Resolver, Query, understand()
understand/slang.py + slang.json	SlangResolver, MoneyResolver; dữ liệu viết tắt
understand/timex.py	TimeResolver, LunarResolver (bọc timeparse, lunar)
understand/basis.py	phân loại FLOW / STOCK
understand/shift.py	ca sáng, ca chiều
understand/coverage.py	so khoảng ngày với phạm vi dữ liệu

File	Vai trò
understand/entity.py	GroundResolver, AmbiguityResolver, che phần đã bị chiếm
normalize.py	norm, Normalized (chuẩn hóa có vị trí)
ground.py	Grounder, phân loại thực thể, NgramIndex, Hint, Group
ingest.py	event_date_range, Business.date_range
context.py	ghép Finding thành các mục của prompt
scripts/mine_slang.py	khai thác ứng viên viết tắt từ train
tests/test_understand.py, test_basis_coverage_entity.py	26 test mới

8. Bài tập tự học

- Thêm một resolver mới: ví dụ nhận biết "top N" trong câu hỏi ("top 3 sp..."). Chỉ cần kế thừa Resolver, trả về Finding, rồi thêm vào danh sách trong context.py.
- Đo độ chính xác của BasisResolver: tự gán nhãn FLOW/STOCK cho 50 câu rồi so với kết quả. Điều chỉnh các regex nếu sai.
- Mở rộng coverage theo từng bảng: câu hỏi về nhập hàng nên so với phạm vi của bảng nhập hàng, không phải toàn shop.
- Sửa ingest để nhận ngày "1 thg 6 2025" (biz_101) và số serial Excel (biz_107); xem phạm vi dữ liệu có hiện ra không.
- Viết resolver sửa lỗi gõ Telex (tieenf, thangs, nawm, hangf...) vì hiện chúng chưa được xử lý.
- Câu hỏi suy nghĩ: vì sao thứ tự ưu tiên là money, lunar, time, slang? Đảo thứ tự thì ví dụ nào hỏng?

Tài liệu này mô tả đúng trạng thái code sau 133 test pass. Chưa có commit nào được tạo.